



University of
Nottingham

UK | CHINA | MALAYSIA

A large, high-resolution image of the Earth as seen from space, showing the Western Hemisphere with North and South America. The Earth is framed by a thin white rectangular border. The background is a deep black space filled with numerous small, distant stars.

COMP-2032
**Introduction to
Image Processing**

Lecture 6B
Hough Transform



Learning Outcomes

IDENTIFY

1. How to find lines?
2. The Hough Transform
3. Other parameter spaces





University of
Nottingham

UK | CHINA | MALAYSIA

Finding Lines: Template Matching



Why Lines?

Geometric primitives make important vision tasks easier

E.g., matching or classifying images

- Some geometric objects are invariant under projection from the (3D) world to (2D) image
- Straight lines in the world create straight lines on the image

A straight line in the image is evidence for a straight line in the world

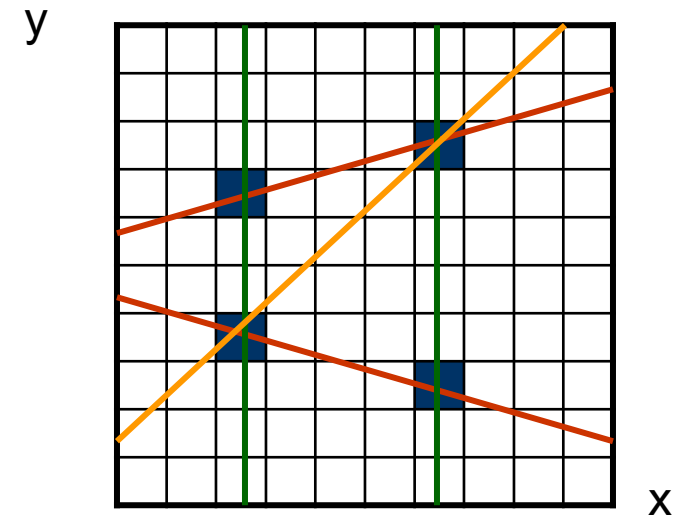


ACK: Prof. Tony Pridmore, UNUK



The Problem

- Suppose straight lines are important
- Edge detection provides a set of points (x_i, y_i) which are likely to lie on those lines
- But how many lines are there and what are their parameters?

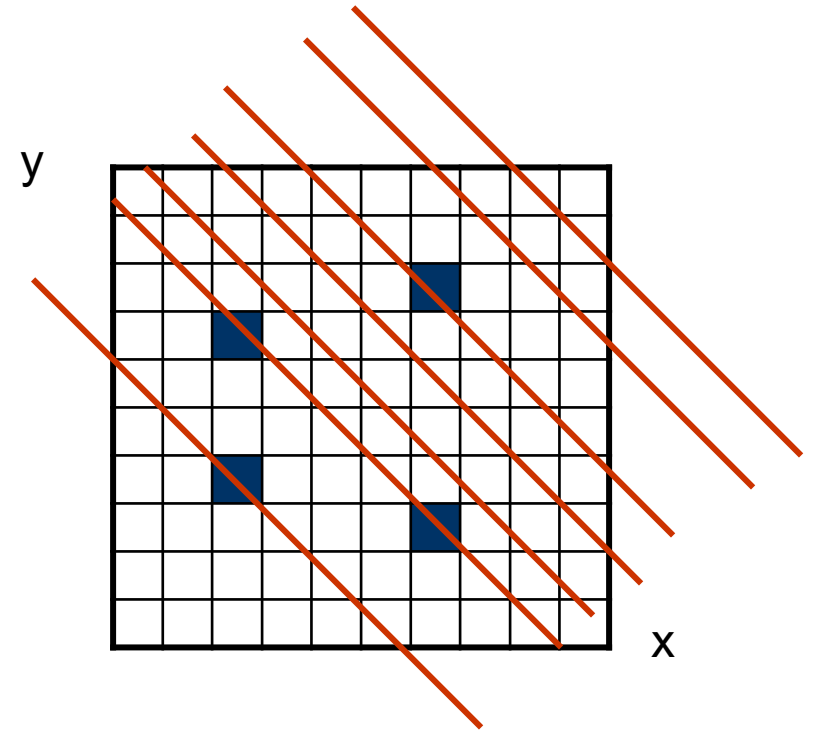




Template Matching

- One solution is to take a straight line and match it to all possible image positions and orientations
 - Compute a measure of fit to the edge data
- E.g., count how many edges are under each possible line

Incredibly expensive!





Template Matching vs. Hough

Classic template matching takes a line, lies it on the image data and asks:

- *Does it fit here?*
- *Here?*
- *Here?*
- *How about here?*
- *Here then?*
-

The Hough takes each data item (edge) and asks:

What lines could pass through this?



University of
Nottingham

UK | CHINA | MALAYSIA

The Hough Transform

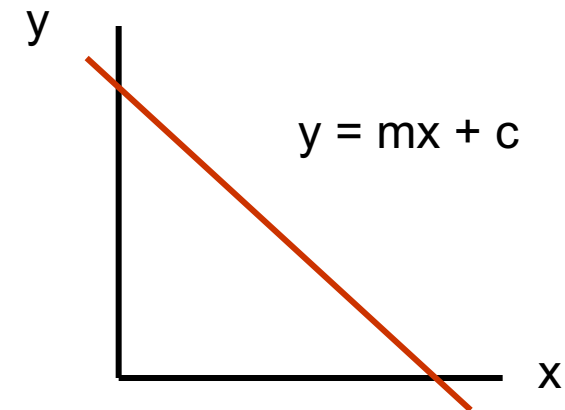


Line Parameters

The standard equation of a line is

$$y = mx + c$$

- If we know the line parameters m , c - we can vary x , compute y and draw the line
- This line represents the set of (x,y) pairs that satisfy the above equation



But we don't know the line parameters – we just have some data points (x_i, y_i)



Parameter Space

But if a data point (x_i, y_i) lies on a line then that line's parameters m, c must satisfy

$$y_i = mx_i + c$$

So we can represent all possible lines through (x_i, y_i) by the set of (m, c) pairs that satisfy this equation

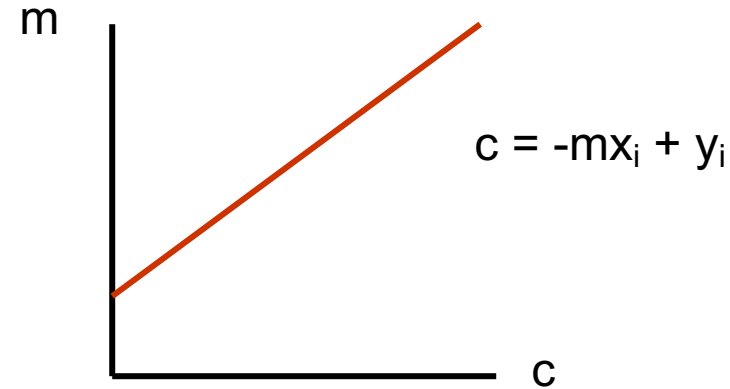
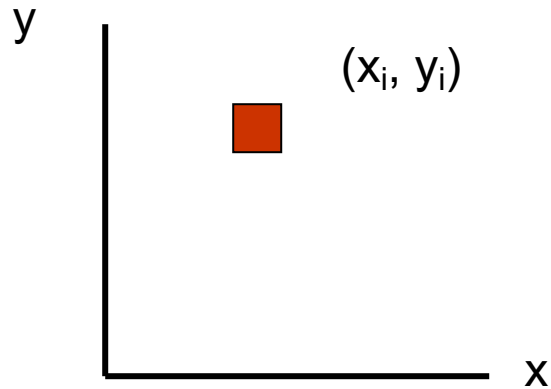
Rearranging the equation gives

$$c = -mx_i + y_i$$

This also describes a straight line, but as x_i and y_i are known and m and c are unknown, that line is in m, c space – a **parameter space**



Parameter Space

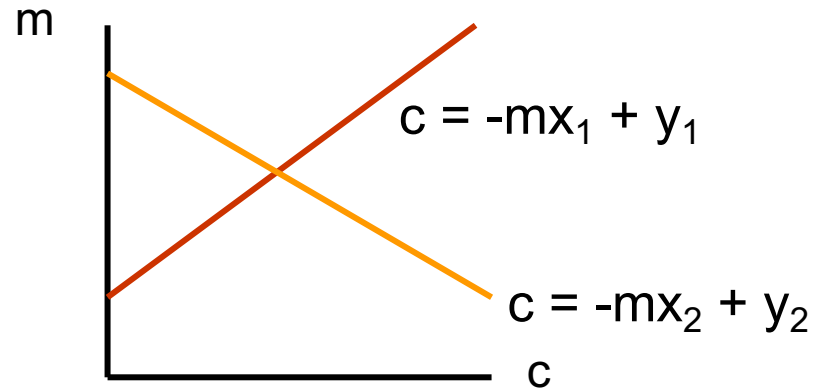
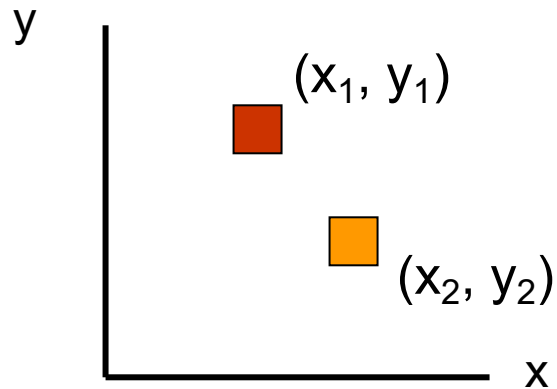


This straight line in m, c space represents all the values of (m, c) that satisfy $y_i = mx_i + c$, and so the parameters of all the lines that could pass through (x_i, y_i)



Parameter Space

Suppose there are 2 edges in our image (x_1, y_1) and (x_2, y_2)

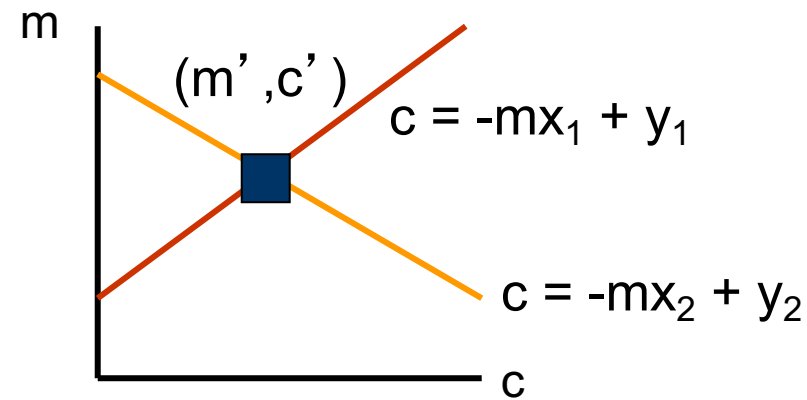
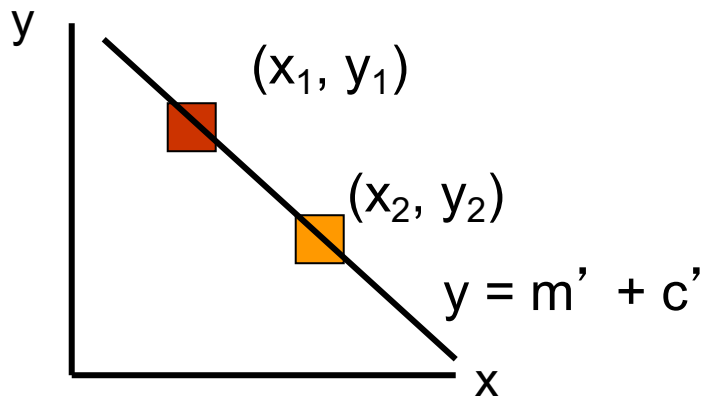


They will each generate a line in (m, c) space representing the set of lines that could pass through them



Parameter Space

The intersection of our 2 lines is the intersection of 2 sets of line parameters



The point of intersection therefore gives the parameters of a line through both (x_1, y_1) and (x_2, y_2)



Parameter Space

Taking this one step further

- All pixels which lie on a line in (x,y) space are represented by lines which all pass through a single point in (m,c) space
- The point through which they all pass gives the values of m' and c' in the equation of the line: $y = m'x + c'$

To detect lines all we need to do is transform each edge point into m,c space and look for places where lots of lines intersect

This is what Hough Transform do



The Hough Transform

1

Quantise (m, c) space into a two-dimensional array A for appropriate steps of m and c

2

Initialise all elements of $A(m, c)$ to zero

3

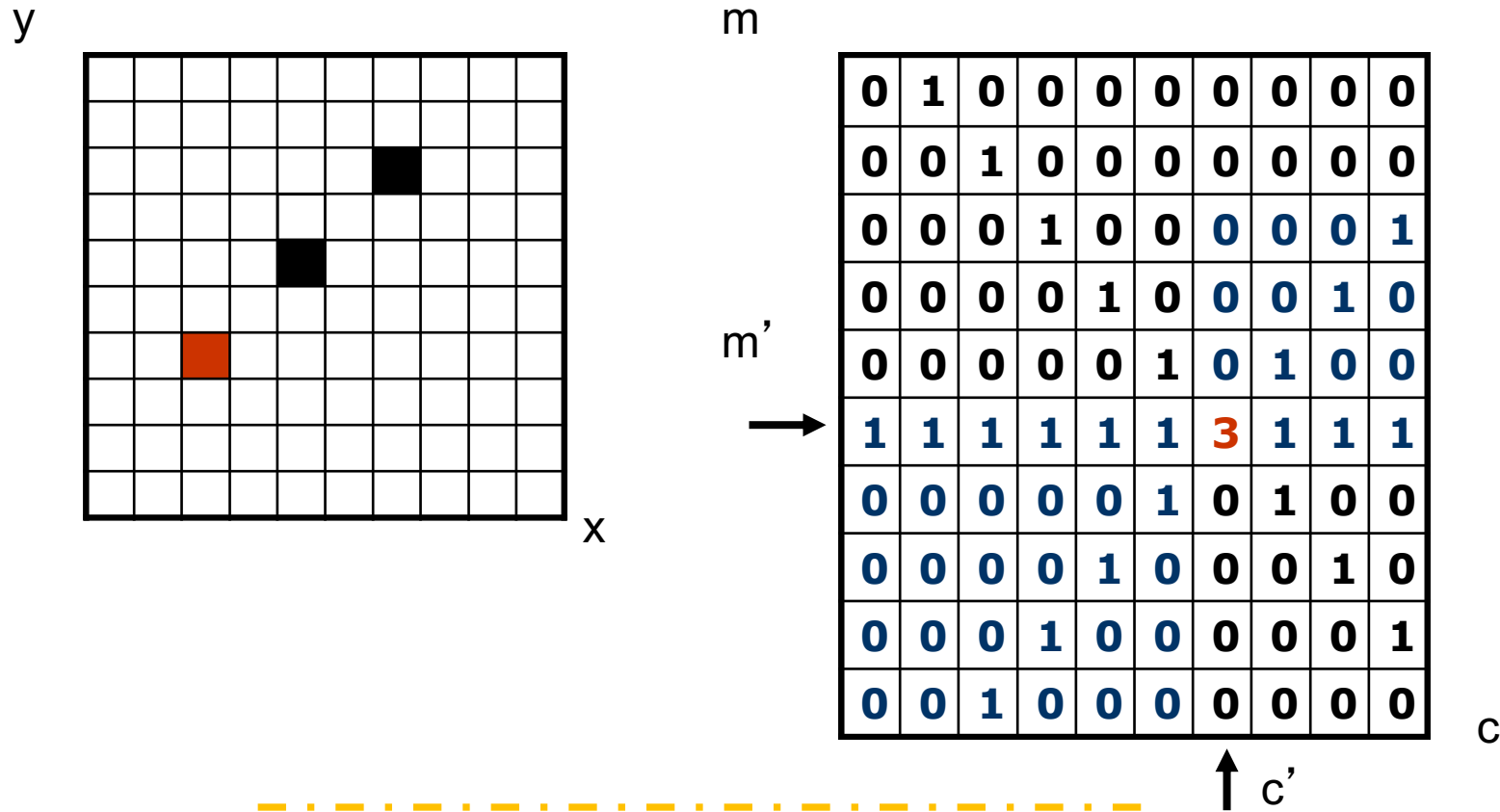
For each pixel (x_i, y_i) which lies on some edge in the image, we add 1 to all elements of $A(m, c)$ whose indices m and c satisfy $y_i = mx_i + c$

4

Search for elements of $A(m, c)$ which have large values – each one found corresponds to a line in the original image



The Hough Transform

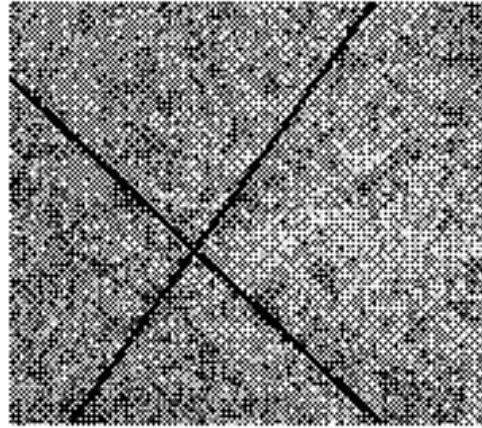


There is one line: $y = m'x + c'$



A (More) Real Example

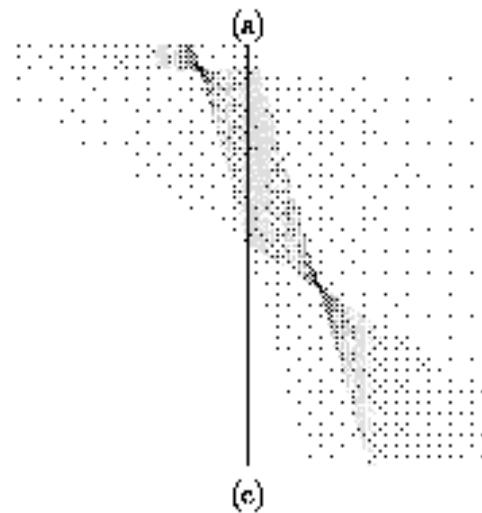
Image



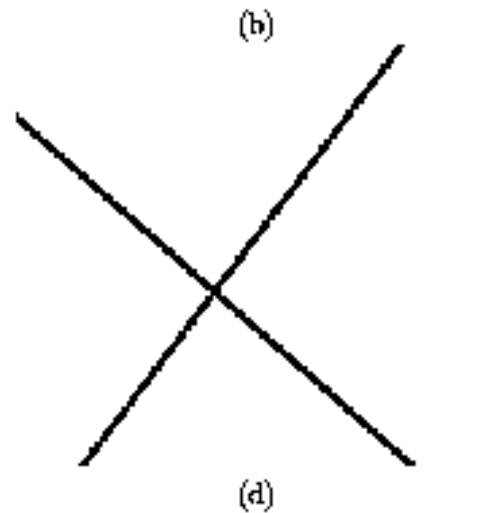
Edges



Accumulators



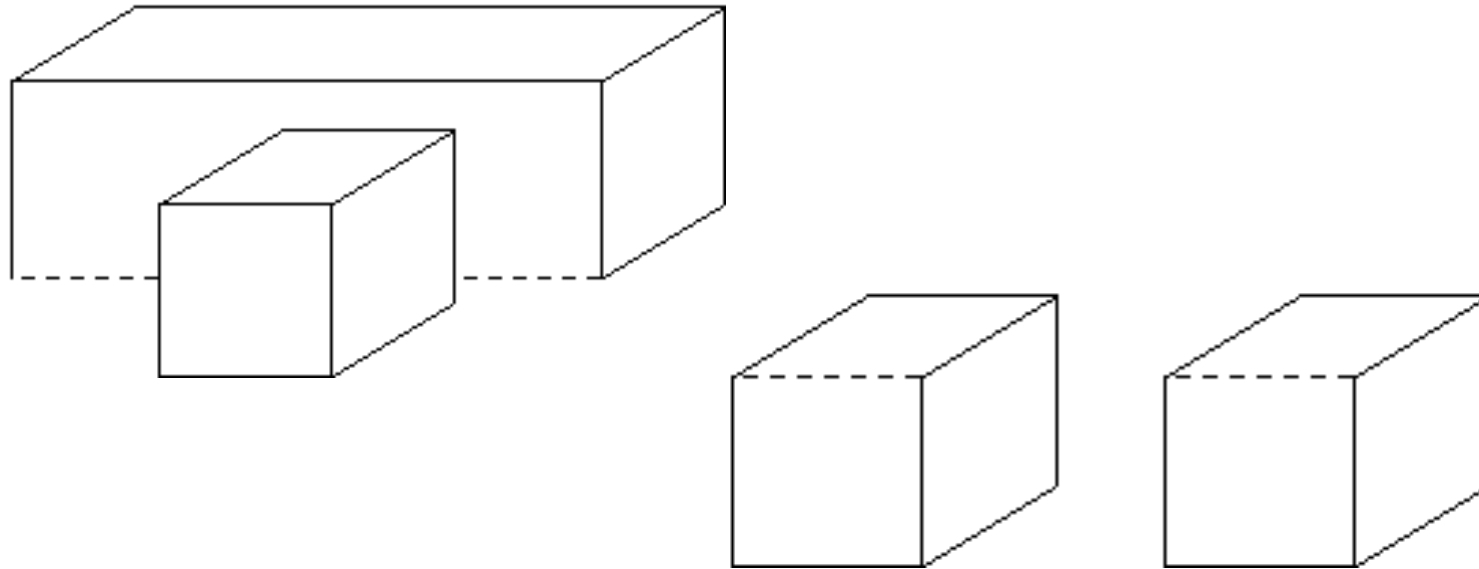
Result





Line Parameters

The Hough Transform only supplies the parameters of the lines it detects; this can be an advantage or a disadvantage





University of
Nottingham

UK | CHINA | MALAYSIA

Other Parameter Spaces



Straight Lines **AGAIN**

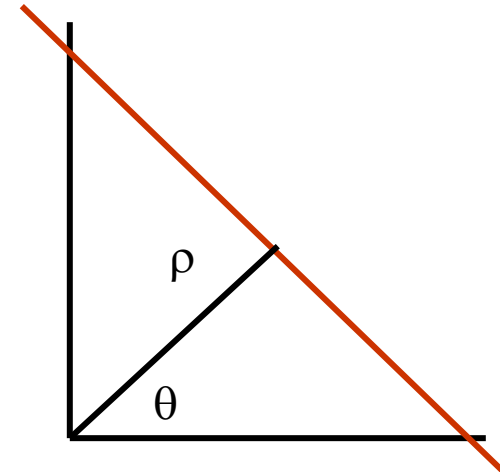
Another equation for a straight line

$$\rho = x.\cos(\theta) + y.\sin(\theta)$$

For an $n \times n$ image

- ρ is in range $[0, 2n]$
- θ is in range $[0, 2\pi]$

Each x_i, y_i generates a sinusoid in ρ, θ space





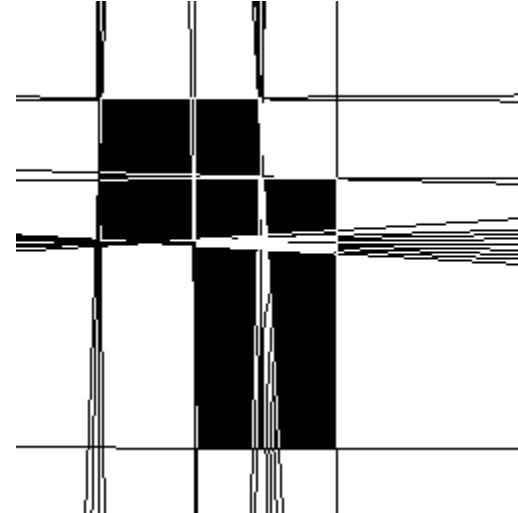
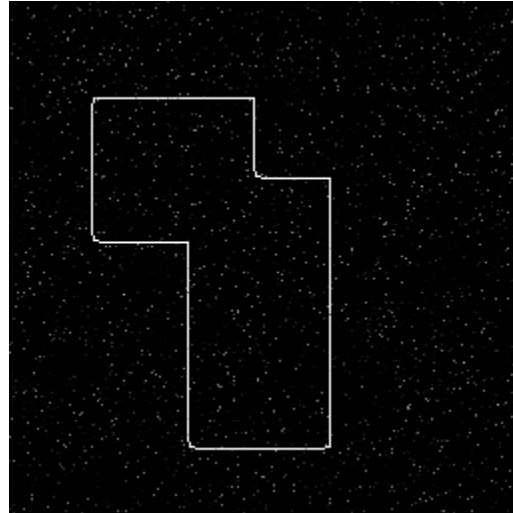
A ρ, θ Example



ACK: Prof. Tony Pridmore, UNUK



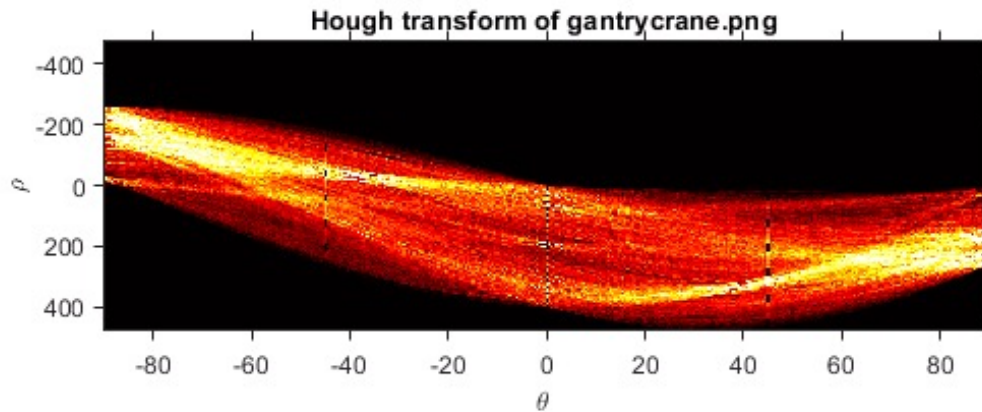
A ρ, θ Example



A key question is what happens to peaks in parameter space when noise or occlusion arise?



The Hough Transform in Python via OpenCV



Like many implementations, OpenCV's Hough allows you to specify which part of the parameter space you're interested in...

[Hough Line Transform](#)

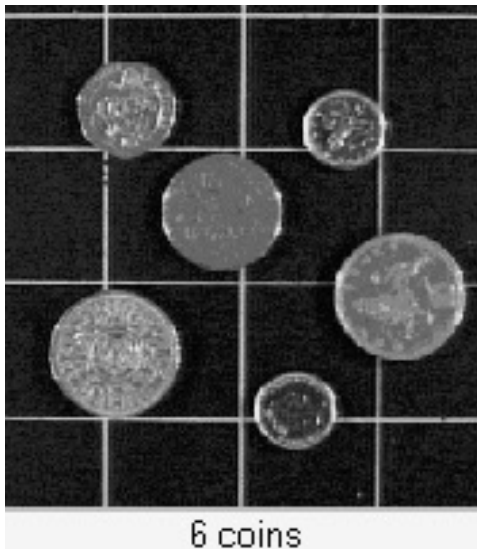
ACK: Prof. Tony Pridmore, UNUK



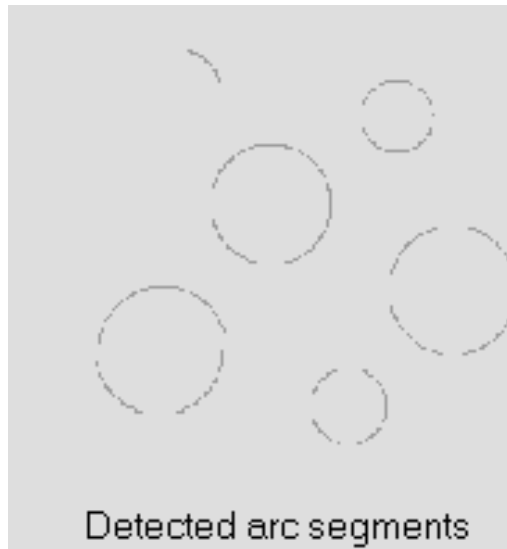
Circles

Parametric equation is $r^2 = (x-a)^2 + (y-b)^2$

- Full problem $\rightarrow$ cones in (a, b, r) space
- Known $r \rightarrow$ circles in (a, b) space



6 coins



Detected arc segments

How about ellipses?



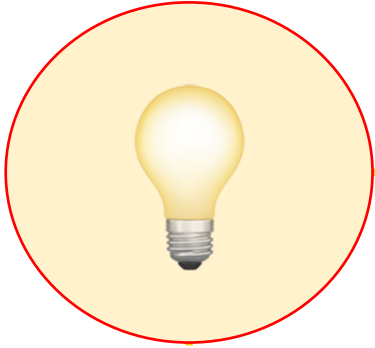
The Hough can in principle be applied to any parameterizable curve, though its not always practical



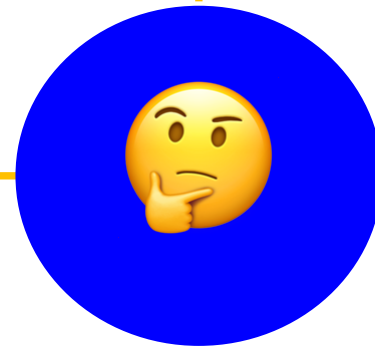
Hough Circle Transform



Summary



1. How to find lines?
2. The Hough Transform
3. Other parameter spaces





University of
Nottingham

UK | CHINA | MALAYSIA

Questions



University of
Nottingham

UK | CHINA | MALAYSIA

NEXT:

Segmentation